RExercise#2_Porras

Renz Benedict Porras

2026-09-30

Part A- Arithmetic Operations

```
# TODO: Store 17 in a variable called a, and 5 in a variable called b
# Then print the results of: a + b, a - b, a * b, a / b

a <- 17
b <- 5

print(a + b)
```

```
## [1] 22
```

```
print(a - b)
```

```
## [1] 12
```

```
print(a * b)
```

```
## [1] 85
```

```
print(a / b)
```

```
## [1] 3.4
```

```
# TODO: Using the same a and b, compute and print a %% b and a %/% b
# a %% b = remainder of a / b
# a %/% b = whole-number (integer) division

print(a%%b)
```

```
## [1] 2
```

```
print(a%/%b)
```

```
## [1] 3
```

```
# TODO: Print 2 raised to the power of 10 using the ^ operator

print(2^10)
```

```
## [1] 1024
```

Question #1: Run 17 / 5, 17 %% 5, and 17 %/% 5 side by side. In your own words, explain how these three results relate to each other mathematically.

```
a <- 17
b <- 5

div <- a / b
mod <- a%%b
quo <- a%/%b

print(div)
```

```
## [1] 3.4
```

```
print(mod)
```

```
## [1] 2
```

```
print(quo)
```

```
## [1] 3
```

Answer number 1: 17 / 5, Divides the numbers completely, returning a decimal. 17 %% 5, Returns only the leftover remainder.17 %/% 5, Returns only the whole number or the quotient, discarding the remainder.

Question #2: Predict the output of 10 %% 3 and 10 %/% 3 before running the code. Were you correct? If not, explain what you misunderstood.

```
print(10 %% 3)
```

```
## [1] 1
```

```
print(10 %/% 3)
```

```
## [1] 3
```

Answer number 2: I did predict it. (%%) is modulo which prints the remainder, while (%/%) prints the whole number or the quotient.

Part B- Basic Data Types: typeof(), class(), and mode()

```
x <- 42
y <- 42L
z <- "42"
w <- TRUE
v <- 3 + 2i
```

```
# TODO: Use typeof() to check the type of each of x, y, z, w, v
typeof(x)
```

```
## [1] "double"
```

```
typeof(y)
```

```
## [1] "integer"
```

```
typeof(z)
```

```
## [1] "character"
```

```
typeof(w)
```

```
## [1] "logical"
```

```
typeof(v)
```

```
## [1] "complex"
```

```
# TODO: Use class() to check the class of each of x, y, z, w, v

class(x)
```

```
## [1] "numeric"
```

```
class(y)
```

```
## [1] "integer"
```

```
class(z)
```

```
## [1] "character"
```

```
class(w)
```

```
## [1] "logical"
```

```
class(v)
```

```
## [1] "complex"
```

```
# TODO: Use mode() to check the class of each of x, y, z, w, v
mode(x)
```

```
## [1] "numeric"
```

```
mode(y)
```

```
## [1] "numeric"
```

```
mode(z)
```

```
## [1] "character"
```

```
mode(w)
```

```
## [1] "logical"
```

```
mode(v)
```

```
## [1] "complex"
```

Question #3): x <- 42 and y <- 42L look almost identical, but typeof(x) and typeof(y) give different answers. Explain the difference in your own words.

Answer number 3: x <- 42 is stored as a double because R defaults to numeric decimals for all numbers, whereas y <- 42L is explicitly stored as an integer because the L suffix forces R to bypass the decimal default and save memory.

Question #4: Is typeof() ever different from class() and from mode() for the same object? Look at your results above and explain any case where this happens (or, if you did not see one, explain what you would expect for a matrix or factor).

Answer number 4: typeof(), class(), and mode() frequently shows different answers for the same object because typeof() identifies the internal data storage type, mode() gives the object's basic type for compatibility with old S language code, and class() represents the object's object-oriented type that dictates how functions treat it.

Part C — Type Conversion (20 min)

```
num_string <- "123"

# TODO: Convert num_string to a numeric value using as.numeric()
# Then check its type with typeof(), class(), and mode()

num_value <- as.numeric(num_string)

as.numeric(num_string)
```

```
## [1] 123
```

```
typeof(num_value)
```

```
## [1] "double"
```

```
class(num_value)
```

```
## [1] "numeric"
```

```
mode(num_value)
```

```
## [1] "numeric"
```

```
# TODO: Convert the number 0 and the number 1 to logical using as.logical()
# What do you get?

as.logical(0)
```

```
## [1] FALSE
```

```
as.logical(1)
```

```
## [1] TRUE
```

```
bad_string <- "hello123"

# TODO: Try converting bad_string to numeric using as.numeric()
# Run this chunk and observe the warning message and the result
as.numeric(bad_string)
```

```
## Warning: NAs introduced by coercion
```

```
## [1] NA
```

Question #5: When you ran the conversion-fail chunk, R produced NA and a warning instead of stopping with an error.

Why do you think R is designed to give a warning here rather than crashing the whole script?

Answer number 5: R replaces the bad data with NA (missing values) so you can fix it later without the whole program stopping.

Question #6: as.logical(0) and as.logical(1) convert to FALSE and TRUE.

What do you think as.logical(2) and as.logical(-1)‘ will produce?

Answer number 6: Both will be TRUE.

```
as.logical(2)
```

```
## [1] TRUE
```

```
as.logical(-1)
```

```
## [1] TRUE
```

Only (0) will be FALSE. Any number either positive or negative will automatically be TRUE.

Part D — Strings, paste(), and paste0()

```
first_name <- "Juan"
last_name <- "Dela Cruz"

# TODO: Print first_name and last_name using print() and/or cat()

cat(first_name, last_name)
```

```
## Juan Dela Cruz
```

```
# TODO: Use paste() to combine first_name and last_name into a full name
# TODO: Use paste0() to do the same, and compare the two results

full_name <- paste(first_name, last_name)
paste0(first_name, last_name)
```

```
## [1] "JuanDela Cruz"
```

```
# TODO: Use paste() with the argument sep = "-" to join first_name and
# last_name with a dash instead of a space

paste(first_name, last_name, sep = "-")
```

```
## [1] "Juan-Dela Cruz"
```

```
age <- 21

# TODO: Use paste0() to build the sentence:
# "Juan Dela Cruz is 21 years old." by combining full_name, age, and text

paste0(full_name, " is ", age, " years old.")
```

```
## [1] "Juan Dela Cruz is 21 years old."
```

Question #7: paste() and paste0() produced different results for the same two strings. Explain exactly why, referencing what each function does by default.

Answer number 7: paste() has a default space between strings, while paste0() doesn't have space so it directly connects strings.

Question #8: In the strings-numbers chunk, you combined a character string with a numeric variable (age) using paste0() without first converting age to a string.

Did this cause an error? Explain what paste0() does internally to make this work, and how it relates to the type conversions you practiced in Part C.

Answer number 8: No, it did not cause an error because paste0() automatically converts numbers into character strings behind the scenes. This automatic process is called implicit coercion, which performs the exact same type conversion you did manually in Part C using as.character().

Part E — Critical Thinking Challenge

```
# TODO: Predict, then test: what is the result of "5" + 5 in R?
# Run it and read the error message carefully.
"5" + 5
```

```
## Error in `"5" + 5`:
## ! non-numeric argument to binary operator
```

```
# I predicted that the output of this will be error because "5" + 5, is non-numeric to binary operator
```

```
# TODO: Now fix the code above so that "5" + 5 correctly evaluates to 10
# using an appropriate conversion function.
as.numeric("5") + 5
```

```
## [1] 10
```

Question #9: Some programming languages automatically convert "5" + 5 into 10 without an error. R does not.

Based on what you've learned about R's data types in this lab, why do you think R requires an explicit conversion instead of guessing what the programmer meant?

Answer number 9: R forces us to convert types explicitly to prevent silent mistakes. If R tried to guess what we meant, it might accidentally glue the data together as text (like making "55") or change it into the wrong number without telling us. Error means you find the mistakes immediately before they mess up our math or statistics.

Question #10: Suppose a classmate writes a script that reads a column of "numbers" from a text file. The column looks numeric, but when they try sum() on it, R throws an error. Based on everything in this lab (types, typeof()/class()/mode(), conversion, warnings), write a short paragraph explaining what likely went wrong and the steps you'd take to diagnose and fix it.

Answer number 10: To fix this, check the data type using class(data$column), find and remove the text typos, and then use as.numeric() to convert the column back into numbers so sum() can work.

CONCLUSION

This activity covers R's arithmetic operations, basic data types, type conversions, and string manipulation techniques. You will practice using standard mathematical operators, explore how R stores different kinds of data using functions like typeof() and class(), and learn how to safely coerce values between types. The exercises also emphasize how R's strict type enforcement and how to handle warnings and errors during data analysis.